Manual de Instalación

Sistema de Gestión de Ventas e Inventario

Índice

1. Requisitos previos
2. Clonar el repositorio
3. Configurar la base de datos PostgreSQL
4. Configurar variables de entorno
5. Instalar dependencias
6. Ejecutar migraciones y seeders
7. Ejecutar el sistema en modo desarrollo
8. Construir para producción
9. Verificar el correcto funcionamiento
10. Solución de problemas comunes
11. Configuración adicional (opcional)
12. Personalización del sistema
13. Respaldo y mantenimiento

---

1. Requisitos previos

Requisitos del sistema

Componente	Versión mínima	Recomendada
Node.js	18.x	20.x LTS
npm	9.x	10.x
PostgreSQL	13.x	15.x
Python	3.9	3.11
Git	2.30	2.40+
PowerShell	5.1	7.x

Requisitos específicos por subsistema

Frontend (React)

- Node.js 18+ con npm
- Navegador moderno (Chrome, Firefox, Edge)

Backend (Express/Node.js)

- Node.js 18+ con npm
- PostgreSQL con soporte para Sequelize

Analytics (Streamlit)

- Python 3.9+
- pip (gestor de paquetes Python)
- Virtual environment (recomendado)

Preparación del entorno

Windows:

```
# Verificar Node.js
node --version  # Debe mostrar v18.x o superior
npm --version   # Debe mostrar 9.x o superior

# Verificar PostgreSQL
psql --version  # Debe mostrar 13.x o superior

# Verificar Python
python --version  # Debe mostrar 3.9 o superior
pip --version
```

Linux/macOS:

```
# Verificar versiones
node --version
npm --version
psql --version
python3 --version
pip3 --version
```

---

2. Clonar el repositorio

Desde GitHub

```
# Clonar el repositorio principal
git clone https://github.com/RobertoA1/mini-dashboard.git

# Ingresar al directorio
cd mini-dashboard
```

Estructura del proyecto

```
mini-dashboard/
├── src/
│   ├── client/                 # Frontend React
│   │   ├── components/         # Componentes React
│   │   │   ├── Cart/
│   │   │   ├── Dashboard/
│   │   │   ├── Orders/
│   │   │   ├── Products/
│   │   │   └── common/
│   │   ├── services/           # Servicios API (api.js)
│   │   ├── slices/             # Redux slices
│   │   ├── App.js              # Rutas principales
│   │   └── index.js            # Punto de entrada
│   ├── models/                 # Modelos Sequelize
│   │   ├── Producto.js
│   │   ├── ProductImage.js
│   │   ├── ProductAttribute.js
│   │   ├── Categoria.js
│   │   ├── Proveedor.js
│   │   ├── Usuario.js
│   │   ├── Orden.js
│   │   ├── OrdenItem.js
│   │   ├── Envio.js
│   │   └── Pago.js
│   ├── slices/                 # Backend API (Express)
│   │   ├── productos/
│   │   │   ├── productos.routes.js
│   │   │   ├── productos.controller.js
│   │   │   └── productos.service.js
│   │   ├── orders/
```

```
│   │   │   ├── orders.routes.js
│   │   │   ├── orders.controller.js
│   │   │   └── orders.service.js
│   │   ├── dashboard/
│   │   │   └── dashboard.service.js
│   │   └── auth/
│   ├── migrations/             # Migraciones Sequelize
│   ├── seeders/                # Seeders de datos
│   │   ├── 20250101000001-demo-categorias.js
│   │   ├── 20250101000002-demo-proveedores.js
│   │   ├── 20250101000003-demo-productos.js
│   │   └── 20250101000006-demo-electronics.js
│   └── config/
│       └── config.js           # Configuración de base de datos
├── streamlit/                  # Servidor Streamlit
│   ├── app.py                  # Aplicación Streamlit
│   ├── requirements.txt        # Dependencias Python
│   └── README.md
├── .env.example                # Ejemplo de variables de
entorno
├── server.js                   # Servidor Express principal
├── start-streamlit.ps1         # Script PowerShell Streamlit
├── start-all.ps1               # Script para iniciar todo
├── package.json                # Dependencias Node.js
└── README.md
```

---

3. Configurar la base de datos PostgreSQL

3.1 Instalar PostgreSQL

Windows:

1. Descargar instalador desde https://www.postgresql.org/download/windows/
2. Ejecutar instalador (seleccionar pgAdmin y command line tools)
3. Establecer contraseña para usuario `postgres`
4. Puerto por defecto: 5432

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
```

```
sudo systemctl enable postgresql
```

macOS:

```
brew install postgresql
brew services start postgresql
```

3.2 Crear base de datos y usuario

```
# Acceder a PostgreSQL (Windows)
psql -U postgres

# Acceder a PostgreSQL (Linux/macOS)
sudo -u postgres psql

# Crear usuario de la aplicación
CREATE USER dashboard_user WITH PASSWORD 'tu_password_seguro';

# Crear base de datos
CREATE DATABASE mini_dashboard_db OWNER dashboard_user;

# Conceder permisos
GRANT ALL PRIVILEGES ON DATABASE mini_dashboard_db TO
dashboard_user;

# Salir
\q
```

3.3 Verificar conexión

```
psql -U dashboard_user -d mini_dashboard_db -h localhost -p 5432
```

---

4. Configurar variables de entorno

4.1 Crear archivo .env

Copie el archivo de ejemplo y configure los valores:

```
# Windows PowerShell
copy .env.example .env

# Linux/macOS
cp .env.example .env
```

4.2 Configurar variables

Edite el archivo `.env` con la configuración apropiada:

```
# ==============================================
# CONFIGURACIÓN DEL SERVIDOR
# ==============================================
NODE_ENV=development
PORT=3000
API_BASE_URL=http://localhost:3000

# ==============================================
# CONFIGURACIÓN DE BASE DE DATOS
# ==============================================
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mini_dashboard_db
DB_USER=dashboard_user
DB_PASSWORD=tu_password_seguro
DB_DIALECT=postgres

# ==============================================
# CONFIGURACIÓN DE SEGURIDAD
# ==============================================
JWT_SECRET=tu_secreto_jwt_muy_seguro_cambiar_en_produccion
JWT_EXPIRES_IN=24h
BCRYPT_ROUNDS=10

# ==============================================
# CONFIGURACIÓN DE STREAMLIT
# ==============================================
```

```
STREAMLIT_URL=http://localhost:8501
API_BASE_URL_STREAMLIT=http://localhost:3000

# ============================================
# CONFIGURACIÓN DE EMAIL (opcional)
# ============================================
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=tu_email@gmail.com
SMTP_PASSWORD=tu_password_app
EMAIL_FROM=Sistema de Ventas <no-reply@tuempresa.com>

# ============================================
# CONFIGURACIÓN DE PAGOS (opcional)
# ============================================
PAYMENT_GATEWAY_KEY=tu_api_key
PAYMENT_GATEWAY_SECRET=tu_api_secret
```

4.3 Seguridad importante

⚠️ **NUNCA suba el archivo `.env` a GitHub:**

```
# Verificar que .env esté en .gitignore
cat .gitignore | grep -E "^\.env"
```

Si no está, agréguelo:

```
.env
.env.local
.env.*.local
```

---

5. Instalar dependencias

5.1 Instalar dependencias del proyecto principal

```
# En la raíz del proyecto
npm install
```

Esto instalará:

- Express y middleware
- Sequelize y pg (PostgreSQL)
- React y dependencias frontend
- JWT, bcrypt, cors, dotenv

5.2 Instalar dependencias de Streamlit

```
# Navegar al directorio streamlit
cd streamlit

# Crear entorno virtual (recomendado)
python -m venv venv

# Activar entorno virtual
# Windows:
venv\Scripts\activate
# Linux/macOS:
source venv/bin/activate

# Instalar dependencias
pip install -r requirements.txt
```

Contenido de `requirements.txt`:

```
streamlit==1.28.0
plotly==5.18.0
pandas==2.1.0
requests==2.31.0
```

5.3 Verificar instalación

```
# Verificar dependencias Node
npm list express sequelize react react-dom

# Verificar dependencias Python (dentro de venv)
pip list | grep -E "streamlit|plotly|pandas"
```

---

6. Ejecutar migraciones y seeders

6.1 Configurar Sequelize CLI

```
# Instalar Sequelize CLI globalmente
npm install -g sequelize-cli

# Verificar instalación
sequelize --version
```

6.2 Ejecutar migraciones

```
# Crear tablas en la base de datos
npx sequelize-cli db:migrate
```

Esto creará:

- `categorias`
- `proveedores`
- `productos`
- `product_images`
- `product_attributes`
- `usuarios`
- `ordenes`
- `orden_items`
- `envios`
- `pagos`

6.3 Ejecutar seeders (datos de prueba)

```
# Ejecutar seeders en orden
npx sequelize-cli db:seed --seed
20250101000001-demo-categorias.js
npx sequelize-cli db:seed --seed
20250101000002-demo-proveedores.js
npx sequelize-cli db:seed --seed
20250101000003-demo-productos.js
npx sequelize-cli db:seed --seed
20250101000006-demo-electronics.js
```

O ejecutar todos:

```
npx sequelize-cli db:seed:all
```

6.4 Verificar datos insertados

```
# Conectar a PostgreSQL
psql -U dashboard_user -d mini_dashboard_db

# Verificar productos
SELECT COUNT(*) FROM productos;

# Verificar categorías
SELECT nombre FROM categorias;

# Salir
\q
```

---

7. Ejecutar el sistema en modo desarrollo

7.1 Método 1: Iniciar todo con PowerShell (Windows)

```
# En la raíz del proyecto, ejecutar el script
.\start-all.ps1
```

Este script inicia:

1. Servidor Node.js (Express) en puerto 3000
2. Servidor Streamlit en puerto 8501

7.2 Método 2: Iniciar servidores manualmente

Terminal 1 - Servidor Node.js:

```
# En la raíz del proyecto
npm run dev
```

Terminal 2 - Streamlit:

```
# En la raíz del proyecto
.\start-streamlit.ps1

# O manualmente:
cd streamlit
venv\Scripts\activate
streamlit run app.py --server.port=8501
--server.address=localhost --server.headless=true
```

7.3 Método 3: Scripts individuales

```
// package.json scripts
{
  "scripts": {
    "dev": "concurrently \"npm run server\" \"npm run client\"",
    "server": "nodemon server.js",
    "client": "react-scripts start",
    "build": "react-scripts build",
    "start": "node server.js"
  }
}
```

7.4 Acceder a la aplicación

Servicio	URL	Descripción
Frontend React	http://localhost:3000	Interfaz de usuario
API Backend	http://localhost:3000/api	Endpoints REST
Streamlit	http://localhost:8501	Gráficos y analytics
PostgreSQL	localhost:5432	Base de datos

---

8. Construir para producción

8.1 Construir frontend React

```
# Generar build optimizado
npm run build
```

Esto crea la carpeta `build/` con archivos estáticos optimizados.

8.2 Configurar variables de producción

```
# .env.production
NODE_ENV=production
PORT=80
API_BASE_URL=https://tu-dominio.com
STREAMLIT_URL=https://analytics.tu-dominio.com
```

8.3 Configurar servidor web (Nginx)

```
# /etc/nginx/sites-available/dashboard
server {
    listen 80;
    server_name tu-dominio.com;

    location / {
        root /var/www/mini-dashboard/build;
        try_files $uri /index.html;
    }

    location /api {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

8.4 Iniciar con PM2 (procesos persistentes)

```
# Instalar PM2
npm install -g pm2

# Crear archivo de configuración
cat > ecosystem.config.js << 'EOF'
module.exports = {
  apps: [
    {
      name: 'dashboard-api',
      script: './server.js',
      instances: 1,
      autorestart: true,
      watch: false,
      max_memory_restart: '1G',
      env: {
        NODE_ENV: 'production',
        PORT: 3000
      }
    },
    {
      name: 'dashboard-streamlit',
      script: './streamlit/venv/bin/streamlit',
      args: 'run app.py --server.port=8501',
      cwd: './streamlit',
      instances: 1,
      autorestart: true
    }
  ]
};
EOF

# Iniciar aplicaciones
pm2 start ecosystem.config.js
pm2 save
pm2 startup
```

---

9. Verificar el correcto funcionamiento

9.1 Verificaciones básicas

```
# 1. Verificar API está respondiendo
curl http://localhost:3000/api/productos

# 2. Verificar Streamlit está corriendo
curl http://localhost:8501

# 3. Verificar base de datos
npm run db:check
```

9.2 Pruebas funcionales

Funcionalidad	URL de prueba	Resultado esperado
Listar productos	GET /api/productos	JSON con array de productos
Dashboard KPIs	GET /api/dashboard/kpis	JSON con métricas
Streamlit charts	http://localhost:8501/?chart=sales-line	Gráfico de ventas
Login	POST /api/auth/login	Token JWT

9.3 Verificar logs

```
# Logs de Node.js
tail -f logs/app.log

# Logs de Streamlit
streamlit logs

# Logs de PostgreSQL
sudo tail -f /var/log/postgresql/postgresql-15-main.log
```

---

10. Solución de problemas comunes

10.1 Error: "No se puede conectar a la base de datos"

Causas posibles:

- PostgreSQL no está corriendo

- Credenciales incorrectas en `.env`
- Firewall bloqueando puerto 5432

Solución:

```
# Verificar PostgreSQL está corriendo
# Windows:services.msc → PostgreSQL → Iniciar
# Linux:
sudo systemctl status postgresql
sudo systemctl start postgresql

# Verificar credenciales
psql -U dashboard_user -d mini_dashboard_db -h localhost -W

# Verificar puerto
netstat -an | grep 5432
```

10.2 Error: "process is not defined" en React

Causa: Uso de `process.env` en código de browser

Solución:

```
// Incorrecto
const url = process.env.REACT_APP_STREAMLIT_URL;

// Correcto
const url = window.location.hostname === 'localhost'
    ? 'http://localhost:8501'
    : 'https://tu-url.com';
```

10.3 Error: Streamlit no muestra gráficos

Verificar:

```
# 1. Verificar Streamlit está corriendo
curl http://localhost:8501

# 2. Verificar API_BASE_URL en streamlit/app.py
# Debe coincidir con el puerto del servidor Node.js
```

```
# 3. Verificar CORS
# En desarrollo, los orígenes deben coincidir
```

10.4 Error: "column does not exist" en seeders

Solución:

```
# Verificar migraciones aplicadas
npx sequelize-cli db:migrate:status

# Si falta alguna, ejecutar
npx sequelize-cli db:migrate

# O reiniciar completamente
npx sequelize-cli db:migrate:undo:all
npx sequelize-cli db:migrate
```

10.5 Puerto 3000 o 8501 ocupado

```
# Encontrar proceso usando el puerto
# Windows:
netstat -ano | findstr :3000
taskkill /PID <ID> /F

# Linux/macOS:
lsof -i :3000
kill -9 <PID>

# O usar puerto alternativo
PORT=3001 npm run dev
```

---

11. Configuración adicional (opcional)

11.1 Configurar SSL/HTTPS

```
# Generar certificado autofirmado (desarrollo)
```

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem
-days 365 -nodes

# En producción, usar Let's Encrypt
certbot --nginx -d tu-dominio.com
```

11.2 Configurar backup automático de base de datos

```
# Script de backup
#!/bin/bash
pg_dump -U dashboard_user mini_dashboard_db > backup_$(date
+%Y%m%d).sql
```

11.3 Configurar monitoreo

```
# Instalar herramientas de monitoreo
npm install -g nodemon

# O usar PM2 con monitoreo
pm2 install pm2-server-monit
pm2 install pm2-logrotate
```

---

12. Personalización del sistema

12.1 Cambiar logo y branding

```
src/client/assets/
├── logo.png          # Logo principal
├── favicon.ico       # Ícono del navegador
└── banner.jpg        # Banner del sitio
```

12.2 Configurar moneda y formato

Editar `src/client/utils/formatters.js`:

```
export const formatCurrency = (value) => {
  return new Intl.NumberFormat('es-PE', {
    style: 'currency',
    currency: 'PEN'
  }).format(value);
};
```

12.3 Agregar nuevas categorías

1. Insertar en base de datos:

```
INSERT INTO categorias (nombre, fecha_creacion)
VALUES ('Nueva Categoría', NOW());
```

1. O crear nuevo seeder

---

13. Respaldo y mantenimiento

13.1 Respaldo de base de datos

```
# Respaldo completo
pg_dump -U dashboard_user mini_dashboard_db > backup_$(date
+%Y%m%d_%H%M%S).sql

# Respaldo solo datos
pg_dump -U dashboard_user --data-only mini_dashboard_db >
backup_data.sql

# Respaldo solo estructura
pg_dump -U dashboard_user --schema-only mini_dashboard_db >
backup_schema.sql
```

13.2 Restaurar base de datos

```
# Crear base de datos nueva
```

```
psql -U postgres -c "CREATE DATABASE mini_dashboard_restore;"

# Restaurar
psql -U dashboard_user -d mini_dashboard_restore <
backup_20240101.sql
```

13.3 Actualización del sistema

```
# 1. Respaldar
git stash
pg_dump ... > backup.sql

# 2. Actualizar código
git pull origin main

# 3. Actualizar dependencias
npm install
pip install -r streamlit/requirements.txt --upgrade

# 4. Ejecutar nuevas migraciones
npx sequelize-cli db:migrate

# 5. Reiniciar servicios
pm2 restart all
```

13.4 Limpieza de logs

```
# Rotación de logs
pm2 logrotate

# Limpieza manual
rm -rf logs/*.log
pm2 flush
```

Contacto y soporte

Para reportar problemas o solicitar ayuda:

- **Repositorio GitHub:** https://github.com/RobertoA1/mini-dashboard/issues

- **Email:** requezadaro@unitru.edu.pe

Versión del manual: 1.0

Última actualización: Abril 2026

Compatible con: Sistema v2.0+